

UNIVERSITY OF ASIA PACIFIC
DEPARTMENT OF CSE

Lab Report - 2

COURSE CODE: CSE 402.

COURSE TITLE: OPERATING SYSTEM LAB.

SUBMITTED BY:

NAME : SYED MAHATASIN HUSSAIN.

STUDENT ID : 23101140

SEMESTER : 4th YEAR 1st SEMESTER

SECTION : C - 2

DATE : 09 - 08 - 2026

SUBMITTED TO

AITA RAHMAN ORTHI

LECTURER AT UAP

1. Problem:

Implement the **Shortest Job First (SJF)** and **First Come First Serve (FCFS)** CPU Scheduling Algorithms for the following processes:

Process	Arrival Time (AT)	Burst Time (BT)
P1	3	3
P2	2	5
P3	5	4
P4	1	3
P5	6	2

For both **SJF (Non-Preemptive)** and **FCFS**, calculate:

- **Completion Time (CT)**
- **Turnaround Time (TAT)**
- **Waiting Time (WT)**
- **Average Turnaround Time (AVG TAT)**
- **Average Waiting Time (AVG WT)**

Also, represent the **process execution sequence** for both algorithms and compare their **Average TAT** and **Average WT** to determine which scheduling algorithm performs better for the given processes.

2. Source Code:

```
# Shortest Job First (SJF)


process = ["p1", "p2", "p3", "p4", "p5"]
at = [3, 2, 5, 1, 6]
bt = [3, 5, 4, 3, 2]

n = len(process)

ct = [0] * n
wt = [0] * n
done = [0] * n
tat = [0] * n
```

```
time = 0
completed = 0

while completed < n:

    x = -1
    for i in range(n):

        if at[i] <= time and done[i] == 0:

            if x == -1 or bt[i] < bt[x]:
                x = i

    if x == -1:
        time += 1

    else:
        time = time + bt[x]

        ct[x] = time
        done[x] = 1
        completed += 1


for i in range(n):
    tat[i] = ct[i] - at[i]
    wt[i] = tat[i] - bt[i]


print("Process\tAT\tBT\tCT\tTAT\tWT")

for i in range(n):
    print(process[i], "\t", at[i], "\t", bt[i], "\t", ct[i],
"\t", tat[i], "\t", wt[i])
```

```
sjf_avg_tat = sum(tat)/n
sjf_avg_wt = sum(wt)/n

print("\nAverage TAT =", sjf_avg_tat)
print("Average WT =", sjf_avg_wt)
print("\n")

# First Come First Serve(FCFS)
lst = [
    ["P1",3,3],
    ["p2",2,5],
    ["p3",5,4],
    ["p4",1,3],
    ["p5",6,2]
]

lst.sort(key=lambda x: x[1])


ct = []
tat = []
wt = []

time = 0

for i in lst:
    name, at, bt = i

    if time < at + 1 :
        time = at

    time += bt
    ct.append(time)

    turnaround = time - at
    tat.append(turnaround)
```

```
    waiting = turnaround - bt
    wt.append(waiting)

print("P_id\t AT\t BT \t CT \t TAT \t WT")

for i in range(len(lst)):
    print(lst[i][0], "\t",
          lst[i][1], "\t",
          lst[i][2], "\t",
          ct[i], "\t",
          tat[i], "\t",
          wt[i], "\t")
print()
avg_tat = sum(tat) / len(tat)
print("Average Tranaround time: ",avg_tat)

avg_wt = sum(wt)/ len(wt)
print("Average Waiting time: ", avg_wt)

print("\n")

if sjf_avg_tat < avg_tat:
    print("SJF is better TAT then FCFS")

else:
    print("FCFS is better TAT then SJF")

if sjf_avg_wt < avg_wt:
    print("SJF is better WT then FCFS")

else:
    print("FCFS is better WT then SJF")
```

3. Output:

```
PS C:\Users\hp\Desktop\CSE 402 Lab\Lab 3> & "C:\Program Files\
Process AT      BT      CT      TAT     WT
p1      3       3       7       4       1
p2      2       5       18      16      11
p3      5       4       13      8       4
p4      1       3       4       3       0
p5      6       2       9       3       1

Average TAT = 6.8
Average WT = 3.4


P_id    AT      BT      CT      TAT     WT
p4      1       3       4       3       0
p2      2       5       9       7       2
P1      3       3       12      9       6
p3      5       4       16      11      7
p5      6       2       18      12      10

Average Tranaround time:  8.4
Average Waiting time:  5.0


SJF is better TAT then FCFS
SJF is better WT then FCFS
PS C:\Users\hp\Desktop\CSE 402 Lab\Lab 3>
```

4. Discussion:

In this experiment, two CPU scheduling algorithms, **Shortest Job First (SJF)** and **First Come First Serve (FCFS)**, were implemented and compared using the same set of processes. The main performance parameters were **Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT)**.

During implementation, some issues were identified in the FCFS algorithm. The process list needed to be sorted according to **Arrival Time**, so lst.sort(key=lambda x: x[1]) was used. The CPU idle-time condition was also corrected to ensure that the CPU waits until a process arrives. After solving these issues, both algorithms produced valid scheduling results.

For the given input, SJF achieved an **average TAT of 6.8** and an **average WT of 3.4**, while FCFS achieved an **average TAT of 8.4** and an **average WT of 5.0**. Therefore, SJF performed better than FCFS for this particular set of processes because it produced lower average waiting and turnaround times.

.

5. Conclusion:

In this experiment, **SJF and FCFS CPU scheduling algorithms** were implemented and compared using the same set of processes. Some issues in the FCFS code, such as sorting processes by Arrival Time and handling CPU idle time, were identified and corrected. The results showed that SJF achieved a lower **Average TAT (6.8)** and **Average WT (3.4)** compared to FCFS, which achieved **Average TAT (8.4)** and **Average WT (5.0)**. Therefore, SJF performed better for this particular dataset because it reduces the average waiting and turnaround time. However, SJF may cause starvation for longer processes, while FCFS is simpler and follows the arrival order.

6. GitHub:

https://github.com/Rizon40/CSE-402-Operating-System-Lab